المدرسة الوطنية للعلوم التطبيقية - القنيطرة
National School of Applied Sciences - Kenitra

Java

Project Report

MediFlow AI

Intelligent Patient Queue Management System

Realized by:
Salma BOUKA
Youssef SADIQUI
Oussama M'SAAD

Supervised by:
Pr. Mehdia AJANA

Academic Year 2025 – 2026

Abstract

*This report documents the design, architecture, and implementation of **MediFlow AI**, a decoupled client-server queue system engineered to optimize patient routing in medical facilities. By applying a multi-factor priority scoring algorithm that mitigates low-acuity starvation, the platform replaces outdated First-In-First-Out (FIFO) schedules with smart, dynamic scheduling metrics. The system achieves high responsiveness through non-blocking asynchronous operations, provides transparent wait-time estimations cloistered per medical practitioner, and ensures data confidentiality via automated GDPR-compliant structural masking layers.*

Keywords: Decoupled Architecture, Spring Boot, JavaFX, Priority Queue Analytics, Heuristic Scheduling, Healthcare IT.

Contents

List of Figures

List of Tables

1. Introduction & Project Scope

1.1 Project Overview

Modern healthcare ecosystems face unprecedented operational strain, with clinical environments worldwide experiencing higher rates of patient influx and resource shortages. Among these challenges, waiting room congestion and inefficient patient routing stand out as critical bottlenecks that directly degrade both clinical outcomes and the overall quality of patient care. When a medical facility is unable to manage patient wait times effectively, it risks amplifying patient anxiety, increasing workplace stress for healthcare personnel, and, in severe cases, delaying vital treatment for critical conditions.

To address these systemic structural inefficiencies, **MediFlow AI** has been developed as an enterprise-grade, decoupled client-server patient queue management platform. Designed to streamline clinical workflows from initial check-in to consultation wrap-up, the system operates as a distributed architecture consisting of a high-throughput Spring Boot REST API server and an interactive JavaFX rich desktop application. By transforming how patient reception, data orchestration, and clinical room allocation are managed, **MediFlow AI** introduces an intelligent framework that automates patient flow tracking, enhances staff performance, and establishes data transparency across the entire clinic.

1.2 Problem Statement

Traditional healthcare waiting areas rely on obsolete First-In-First-Out (FIFO) queue mechanics or rudimentary manual numbering sheets. While a sequential approach is fair and efficient for commercial or administrative applications, it introduces significant operational failures within a clinical environment due to several core limitations:

1. **Acuity Blindness:** FIFO frameworks treat every patient identically, regardless of their underlying medical condition. A patient presenting with severe acute symptoms (such as sudden chest pain or respiratory distress) is forced to wait behind individuals with low-acuity conditions (such as routine prescription renewals or mild seasonal symptoms) simply based on their time of arrival.

2. **The Low-Acuity Starvation Dilemma:** Conversely, if a system uses a simple static urgency filter where urgent cases always skip the line, low-priority patients can face infinite delays (starvation) during peak hours as new critical walk-ins continually arrive.

3. **Lack of Data Transparency:** Patients sitting in conventional waiting rooms often experience severe psychological anxiety caused by invisible delays. Without real-time insight into their position in line, estimated wait times, or the active

workloads of specific medical offices, patients are left entirely disconnected from the care process.

4. **Staff Cognitive Overload:** Receptionists and nursing staff are frequently forced to handle double bookings, manually cross-reference paper appointment schedules, track down which doctor is available, and manage frustrated crowds, which significantly increases error rates during intake data capture.

1.3 Project Goals & Scope

The primary objective of **MediFlow AI** is to eliminate structural queue bottlenecks by implementing an automated, intelligent scheduling framework. To achieve this engineering objective, the project scope focuses on three core pillars:

- **Multi-role Cross-Boundary System Deployment:** Building and coordinating three separate modules to manage the patient lifecycle seamlessly. This includes the *Receptionist Dashboard* for secure data entry and patient triage, the private *Doctor Dashboard* for managing consultations and background updates, and a read-only *Public Lounge Monitor* that acts as an open TV board to display updated queue positions and keep waiting patients informed.
- **Integration of Rule-Based Prioritization Heuristics:** Developing a dynamic, multi-factor priority scoring algorithm that recalculates queue order in real time. The mathematical engine weights clinical acuity, vulnerabilities (such as senior demographics), and pre-scheduled appointments, while applying a linear anti-starvation coefficient (+1 point per 5 minutes elapsed) to ensure low-acuity tickets advance systematically.
- **Implementation of Reactive, Accessible Human Interfaces:** Designing highly intuitive, interactive user interfaces optimized for speed and clarity. The client presentation layer utilizes client-side Natural Language Processing (NLP) text listeners to automate urgency classification, non-blocking asynchronous UI worker threads (`Platform.runLater`) to eliminate screen freezing, and strict data masking rules to protect patient privacy on public displays.

2. Requirement Analysis & Specifications

2.1 Functional Requirements

Functional requirements define the core operational capabilities, workflows, and specific tasks that **MediFlow AI** must execute to accommodate clinical business goals. The system is structured into three highly specialized functional modules.

2.1.1 Receptionist Operations

The receptionist module acts as the primary data ingestion point for the clinical ecosystem. The intake process requires precise task decomposition to minimize registration latency and eliminate data entry errors:

- **Patient Intake & Data Capture:** The system provides structured entry controls via `TextField` components to collect essential demographic metrics—including full name (`nameInput`), email address (`emailInput`), and age (`ageInput`)—ensuring structural validation before data dispatch.
- **Edge NLP Triage Assistance:** As the user enters the chief complaint into the `reasonInput` field, a real-time text property listener captures token strings to perform localized semantic parsing. Upon identifying pathognomonic keywords (e.g., *"poitrine"*, *"respirer"*, *"sang"*), the system automatically sets the `urgencyInput` dropdown value to `HIGH` and highlights the borders in high-contrast red to notify the operator.
- **Alphanumeric Shortcode Generation:** Rather than exposing confusing millisecond timestamps, the backend engine processes incoming records sequentially to issue clean, distinct shortcodes such as `TCK-001` or `TCK-002` for optimized workplace communication.
- **Secure Dynamic Doctor Assignment:** The receptionist can assign a patient to an active practitioner. Instead of manual typing, clicking the *"Assigner"* button triggers an asynchronous network request to fetch a list of registered doctors from the database, rendering them in a stylized `ComboBox` layout (e.g., *"Dr. Mahdi [Cardiologie]"*) to eliminate typing errors.
- **Absence & Queue Pruning Lifecycle:** If a patient fails to respond when called, the receptionist can mark the entry as `ABSENT`. The system instantly sets this status, updates the database, removes the ticket from active rows, and shifts following lines forward.

2.1.2 Medical Practitioner Operations

The private medical practitioner view is engineered to streamline room management, minimize clinical overhead, and track waiting lines without manual interference:

- **Isolated Queue Boundaries:** Upon a successful login verification via `LoginController`, the system extracts the practitioner's distinct `doctorId` from the thread-safe `SessionContext` singleton. The interface filters out general clinical rows, displaying only the patients explicitly assigned to that doctor.
- **Automated Background Polling Loops:** To keep records perfectly fresh without requiring manual screen refreshes, the controller configures a background thread pool via a `ScheduledExecutorService`. This executor issues a network refresh task every 10 seconds to sync queue changes silently.
- **Consultation Lifecycle Transitions:** Doctors govern care states using responsive action buttons:
  1. *Démarrer Consultation:* Shifts the active ticket status from `WAITING` to `IN_CONSULTATION`, logging the timestamp and updating public monitors.
  2. *Terminer Consultation:* Marks the ticket as `COMPLETED`, captures the departure time, and evaluates the next priority row.

2.1.3 Lounge Display Monitor

The public dashboard provides a read-only, high-visibility interface designed for television screens in the facility's waiting area:

- **Strict Read-Only Bounds:** To maintain system integrity in public spaces, all interactive input controls, buttons, and text fields are omitted. Data updates are pulled purely through a 10-second automatic polling schedule running on an independent background execution pool.
- **GDPR-Compliant Patient Anonymization:** To protect patient privacy in public lounges, the controller runs full name strings through a masking parser before rendering. Names are reformatted to protect identities while remaining recognizable (e.g., *"Jean Dupont"* displays as `J. DUPONT`), satisfying data protection requirements.
- **Dynamic Relative Waiting Windows:** Instead of displaying confusing static numbers, the column calculates the relative remaining wait time on the fly, displaying the count in minutes or flashing an alert like *"Votre tour !"* when the patient is called.

2.2 Non-Functional Requirements

Non-functional requirements describe the structural constraints, performance thresholds, security baselines, and quality metrics that define the overall stability of the system.

- **Performance (Asynchronous HTTP Execution):** To ensure a responsive user experience, the desktop interface must never freeze or experience stuttering during database or network operations. The system achieves this by routing all remote requests through the native Java `java.net.http.HttpClient` using asynchronous handlers (`sendAsync`). This keeps heavy JSON processing on separate worker threads, allowing the main JavaFX Application Thread to focus entirely on rendering.
- **Security (Context Isolation & Session Singletons):** User state information must be managed securely to prevent cross-boundary data leaks or unauthorized role access. Authenticated credentials populate a global, thread-safe `SessionContext` Singleton pattern instance. This context isolates data access by role, applies automated cross-origin controls (`@CrossOrigin`), and completely wipes session caches during a logout sequence by executing a deep `clear()` action.
- **Usability (Visual Signaling, Alerts & State Synchronization):** The interface must remain clean and highly legible from a distance. The system uses conditional row factories to apply distinct, color-coded visual cues (e.g., light red highlighting for `HIGH` acuity rows). Additionally, it provides immediate feedback through embedded progress indicators during background processing and displays explicit verification dialog alerts to minimize errors during high-stress operations.

3. System Modeling & Database Design

3.1 UML Modeling

Unified Modeling Language (UML) diagrams are leveraged to formalize the structural and behavioral specifications of **MediFlow AI**. This object-oriented blueprint ensures strict adherence to requirements before deploying the client-server system components.

3.1.1 Use Case Diagram

The global Use Case Diagram outlines the functional boundaries of the platform, mapping system capabilities to specific actors. The application identifies three primary human actors alongside one system actor:

- **Receptionist (Actor):** Governs initial point-of-care tasks. Responsible for running the check-in process, executing edge NLP triage diagnostics, assigning patients to specific practitioners, and marking un-called profiles as absent.
- **Medical Practitioner / Doctor (Actor):** Interacts exclusively with role-specific workspaces. Responsible for starting active consultations, capturing medical summaries, and finalizing patient records.
- **Administrator (Actor):** Manages high-level data administration, including doctor registrations and the definition of clinical division profiles.
- **Patient Monitor System (System Actor):** Operates as an automated, read-only process that consumes data streams to display structural queues in the waiting room lounge.

As shown in Figure 3.1, access to all role-specific use cases is protected by a core dependency step (`«include»`) tied directly to the central authentication workflow.

3.1.2 Sequence Diagram

The Sequence Diagram documents the step-by-step transactional communication paths that occur across system boundaries during runtime operations. Figure 3.2 illustrates the execution flow for patient creation, priority evaluation, and multi-channel synchronization:

The communication sequence executes across five specific structural steps:

1. **Intake Capture:** The Receptionist inputs demographic values into the JavaFX input fields. The local text listener analyzes the symptom text on the fly, automatically updating the triage level if acute signs are found.

2. **Asynchronous Dispatch:** Clicking *"Ajouter"* transforms the form data into a JSON string via Gson and dispatches it asynchronously using `HttpClient.sendAsync()` to the backend REST endpoint `POST /api/tickets/add`.

3. **Transactional Persistence:** The Spring Boot backend processes the incoming payload. The service layer executes within an atomic transaction (`@Transactional`), saving the entity records to MySQL via the JPA abstraction layer.

4. **Heuristic Evaluation & Notification:** The service layer queries the `PriorityService` engine to compute the ticket's active priority score and queue slot. Once calculated, a thread task calls the `EmailService` layer to send a confirmation notice to the patient via an SMTP secure relay channel.

5. **UI Thread Refresh:** The backend returns an HTTP 200 OK success payload to the client. The client's asynchronous worker pool captures this return signal and uses `Platform.runLater()` to safely pass data modifications to the main JavaFX UI thread, updating the active tables without freezing the screen.

3.1.3 Class Diagram

The Class Diagram structural layout models the object-oriented architecture of the platform. The system uses a clean architectural separation between backend persistence entities and frontend data representation structures.

As mapped in Figure 3.3, the core object hierarchy models healthcare domain relationships through specific aggregation links:

- `Ticket`: The central entity that tracks care interactions. It encapsulates a single `Patient` profile alongside an optional `Doctor` assignment to support targeted medical routing.

- `Doctor`: Represents an active medical practitioner. It maintains a strict $1 : 1$ composition link to a system `User` account for access control, while holding a many-to-one link (`ManyToOne`) to a `MedicalService` division.

- `MedicalService`: Defines a distinct clinical pôle (e.g., Cardiology). It stores default consultation metrics used by the estimation engine to calculate relative waiting windows for active rows.

—

3.2 Database Design

The data layer is engineered to guarantee strict ACID transaction compliance, clean relational storage, and total data integrity across all operational care stages.

3.2.1 Entity-Relationship Diagram (ERD)

The database structure uses foreign key constraints and index linkages to optimize query speeds during high-volume data updates.

The design enforces structural constraints to map entity lifecycles correctly:

- A single entry in the `users` table maps to exactly one practitioner profile in the `doctors` table via a unique foreign key constraint (`user_id`), preventing identity reuse.
- The `medical_services` master table acts as a data parent, linking a single department row to multiple practitioners through a targeted relational index link.
- The transactional `queue_tickets` table acts as the system's analytical engine, joining the `patients` and `doctors` profiles together while logging time logs to support performance analytics.

3.2.2 Data Dictionary

The following tables define the schema attributes, database constraints, index parameters, and types used to configure the MySQL engine inside the Docker network.

Table 3.1: Data Dictionary for the `users` Table

Column Name	Data Type	Null	Key	Description / Constraints
`id`	`BIGINT`	No	PK	Auto-increment primary access key.
`full_name`	`VARCHAR(150)`	No	–	Complete structural name of the account owner.
`email`	`VARCHAR(100)`	No	UK	Unique login email string index.
`password`	`VARCHAR(255)`	No	–	Encrypted account credential payload.
`role`	`VARCHAR(30)`	No	–	Authorization tag: `ADMIN`, `RECEPTIONIST`, `DOCTOR`.
`enabled`	`BOOLEAN`	No	–	State flag managing system access authorization.
`created_at`	`TIMESTAMP`	No	–	Automatic system creation record timestamp.

Table 3.2: Data Dictionary for the `patients` Table

Column Name	Data Type	Null	Key	Description / Constraints
`id`	`BIGINT`	No	PK	Auto-increment primary registration key.
`full_name`	`VARCHAR(150)`	No	–	Full identity name used for anonymization.
`email`	`VARCHAR(100)`	No	–	Notification email address target.
`age`	`INT`	No	–	Quantitative age metric used for vulnerability scoring.

Table 3.3: Data Dictionary for the `queue_tickets` Table

Column Name	Data Type	Null	Key	Description / Constraints
`id`	`BIGINT`	No	PK	Auto-increment transactional tracking ID.
`ticket_number`	`VARCHAR(20)`	No	UK	Unique formatted usage shortcode (e.g., `TCK-001`).
`urgency_level`	`VARCHAR(20)`	No	–	Triage severity flag: `LOW`, `MEDIUM`, `HIGH`.
`has_appointment`	`BOOLEAN`	No	–	Scheduler status marker (+15 bonus points).
`priority_score`	`INT`	No	–	Dynamic calculated scoring matrix total.
`estimated_wait`	`INT`	No	–	Calculated wait time metric in minutes.
`status`	`VARCHAR(30)`	No	–	Lifecycle state: `WAITING`, `IN_CONSULTATION`, etc.
`patient_id`	`BIGINT`	No	FK	Links to target record in the `patients` table.
`doctor_id`	`BIGINT`	Yes	FK	Links to target doctor in the `doctors` table.
`created_at`	`TIMESTAMP`	No	–	Timestamp recording initial arrival time.
`completed_at`	`TIMESTAMP`	Yes	–	Timestamp logging departure time.

Table 3.4: Data Dictionary for the `medical_services` Table

Column Name	Data Type	Null	Key	Description / Constraints
`id`	`BIGINT`	No	PK	Auto-increment clinical primary service ID.
`name`	`VARCHAR(100)`	No	–	Descriptive name of the department division.
`avg_time`	`INT`	No	–	Standard consultation length configuration in minutes.
`active`	`BOOLEAN`	No	–	Operations status flag for the department.

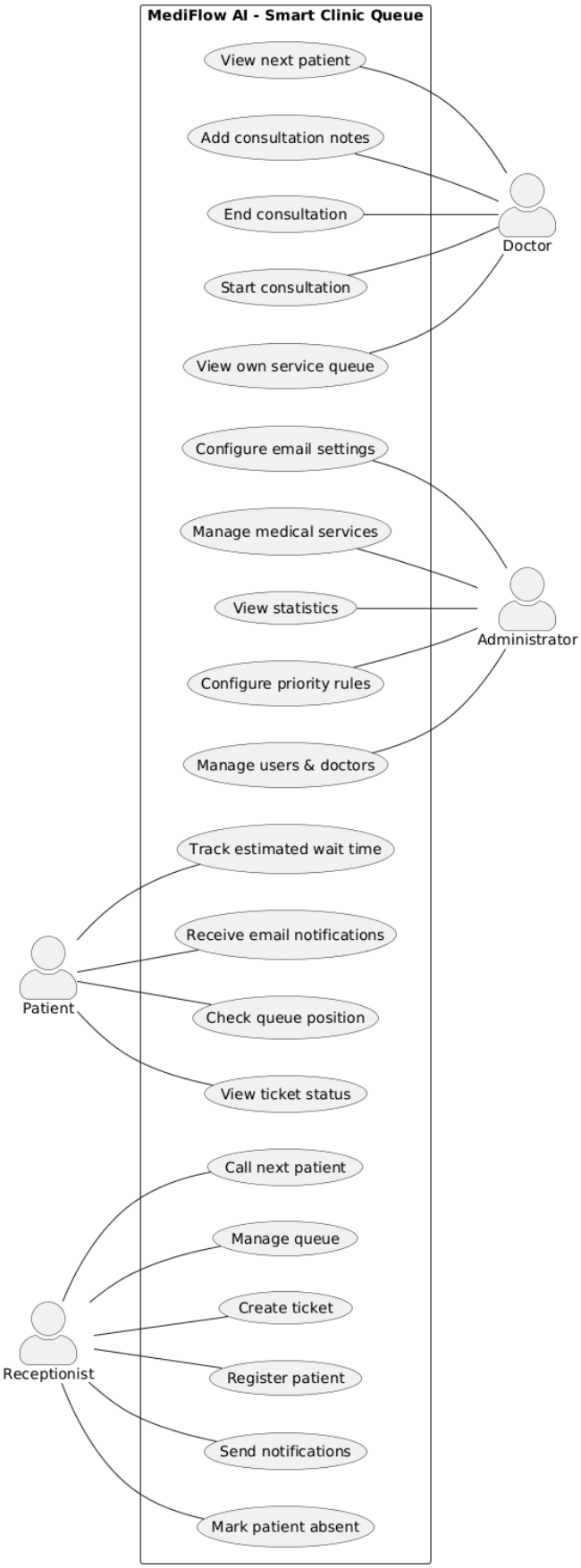

Figure 3.1: MediFlow AI Global Use Case Diagram

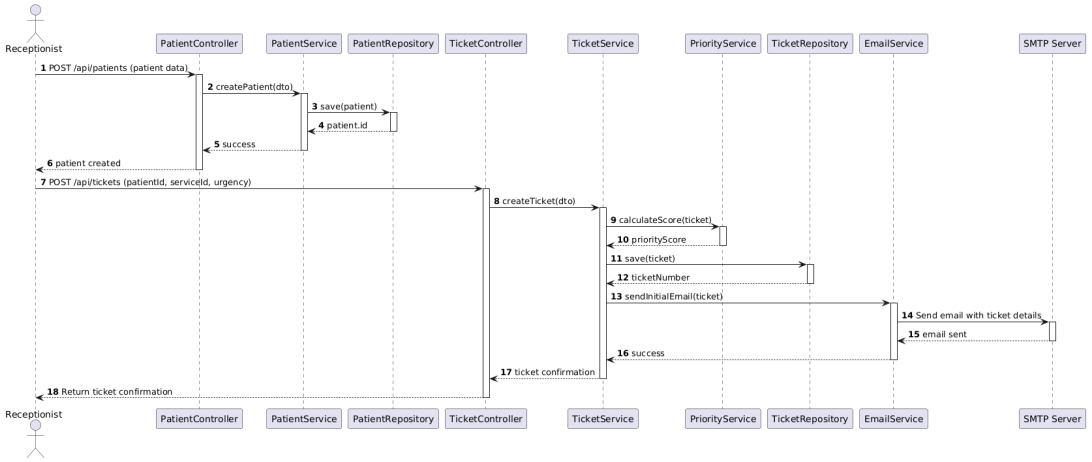

Figure 3.2: Asynchronous Patient Intake and Prioritization Sequence Flow

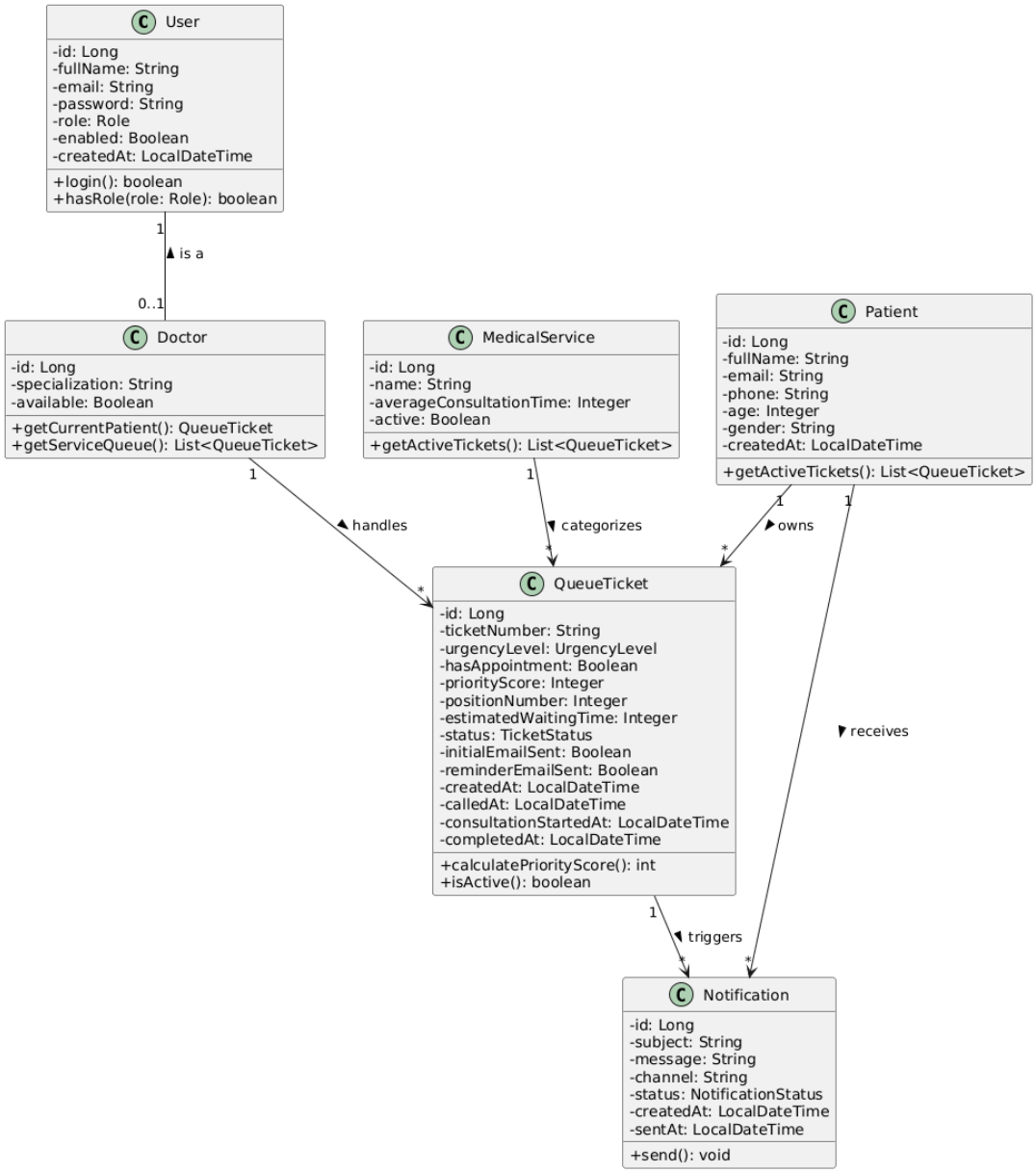

Figure 3.3: MediFlow AI Core Class Model Architecture

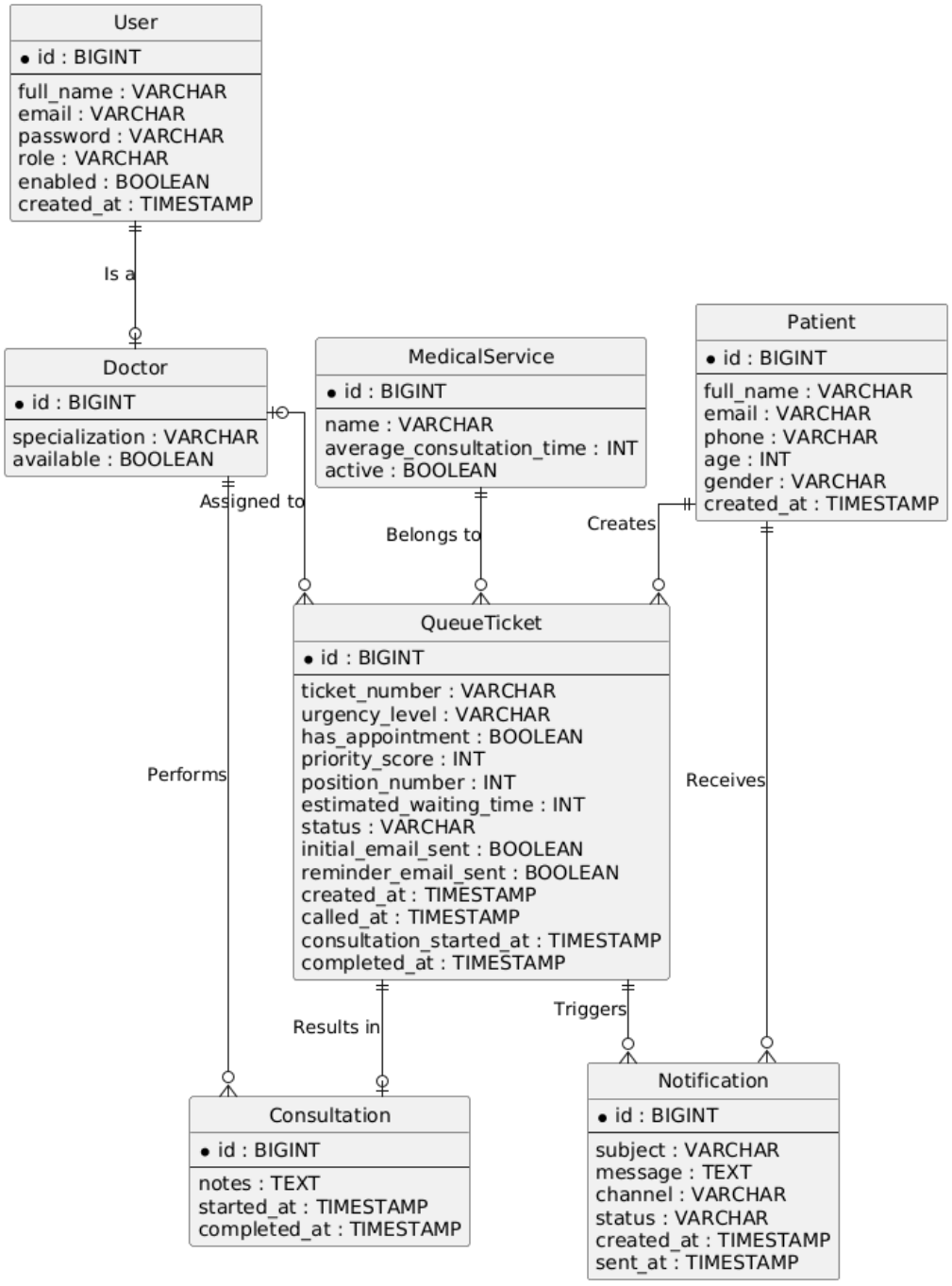

Figure 3.4: Relational Entity-Relationship Diagram Schema Layout

4. Software Architecture & Technical Stack

4.1 Decoupled Client-Server Architecture

MediFlow AI is engineered around a fully decoupled, multi-tier client-server architectural style. By separating the application into an independent backend database manager and a stateful native desktop user interface, the system ensures a strict separation of concerns, high system scalability, and streamlined maintainability.

The core decoupling strategies implemented in this architecture include:

- **Stateless RESTful API Layer:** The Spring Boot application functions exclusively as a high-throughput, stateless computation engine. It exposes deterministic HTTP REST endpoints and does not maintain client session states on the server. This structure allows the database service to handle multiple parallel requests without session overhead.
- **Asynchronous Data Exchange Boundary:** Communication between the client and server occurs through structured, asynchronous HTTP channels. The system exchanges data strictly using lightweight JSON payloads serialized and deserialized via the Google Gson engine, eliminating tight binary dependencies between the layers.
- **Network Isolation and Cross-Origin Protections:** By configuring explicit boundary filters and cross-origin controls (`@CrossOrigin("*" )`), the backend remains fully isolated. This design enables separate deployment environments, allowing the backend to run inside containerized Linux environments while the rich desktop clients execute natively on disparate terminal environments.

4.2 Design Patterns Implemented

To ensure structural maintainability, testability, and adherence to enterprise software design criteria, two primary design patterns are implemented across the codebase.

4.2.1 Model-View-Controller (MVC)

The frontend application layout strictly enforces the Model-View-Controller (MVC) pattern natively supported by JavaFX. This pattern decouples presentation assets from operational controller classes:

1. **The View (FXML / CSS):** Declarative visual designs are isolated inside dedicated XML-based layout files (e.g., `Dashboard.fxml`, `WaitingRoom.fxml`). Presentation layouts are styled using separate cascading style sheets (`style.css`).

2. **The Controller (JavaFX Controllers):** Classes such as `DashboardController` capture operational events, bind user actions, and orchestrate background workers. Components are injected from the view into the controller using the strict encapsulation rules defined in `module-info.java` via the `@FXML` annotation marker.

3. **The Model (POJO Domain Entities):** Client-side data layers use Plain Old Java Objects (POJOs) like `Ticket.java` and `Patient.java`. These models are free from bulky JPA annotations and focus entirely on UI properties, thread-safe data extraction, and structural cell factory bindings.

4.2.2 Singleton Pattern

State synchronization across changing JavaFX scenes requires secure, centralized session tracking. The system achieves this by implementing a thread-safe **Singleton Pattern** inside the `SessionContext.java` utility.

This global container provides immediate access to authenticated user properties—including the `userId`, user `fullName`, authorized `role`, and associated `doctorId`—without requiring complex, error-prone manual variable passing during scene switches.

```
package com.mediflow.ui.util;

public class SessionContext {
  private static SessionContext instance;

  private Long userId;
  private String fullName;
  private String role;
  private Long doctorId;

  private SessionContext() {}

  public static SessionContext getInstance() {
    if (instance == null) {
      instance = new SessionContext();
    }
    return instance;
  }

  public void clear() {
    userId = null;
    fullName = null;
    role = null;
    doctorId = null;
  }

  // Standard getters and setters
  public Long getUserId() { return userId; }
  public void setUserId(Long userId) { this.userId = userId; }
  public String getRole() { return role; }
  public void setRole(String role) { this.role = role; }
  public Long getDoctorId() { return doctorId; }
  public void setDoctorId(Long doctorId) { this.doctorId = doctorId; }
}
```

Listing 4.1: Thread-Safe Session Context Singleton Implementation

Executing the `clear()` method during logouts ensures total environment purge, preventing session hijacking or unauthorized data exposure between shifting operators.

4.3 Technical Stack Reference

The implementation relies entirely on a collection of modern frameworks, language engines, and deployment environments selected to fulfill the project's non-functional performance, safety, and stability requirements.

Table 4.1: MediFlow AI Comprehensive Technical Stack Matrix

Layer / Domain	**Component Technology**	**Version / Framework**
Backend API Framework	Spring Boot Development Suite	Version 4.0.6
Language Environment	OpenJDK Runtime Environment	Java 17 LTS
Data Access Abstraction	Spring Data JPA Core Engine	Hibernate ORM 7.2.12
Relational Data Engine	MySQL 8.0 Community Server	Containerized Docker Image
Presentation Layout Layer	OpenJFX Rich Client Platform	JavaFX 17 / 13
JSON Parsing Architecture	Google Gson Serialization Engine	Version 2.10.1
Asynchronous Network Layer	Native Java HTTP Handler	`java.net.http.HttpClient`
Notification Relay Relay	Jakarta Mail Sender Protocol	SMTP Secure Transport TLS
Deployment Orchestration	Multi-Stage Docker Tooling	Docker Compose Network

5. UI/UX Design & Implementation

5.1 Interface Layout Strategies & Space Optimization

The interface layer of **MediFlow AI** is engineered to address the fast-paced, high-stress realities of clinical environments. To minimize cognitive fatigue and reduce human error, the interface layout relies on a structured, multi-pane architecture powered by JavaFX layouts and stylized cascading stylesheets (`style.css`).

Three primary interface strategies were applied to maximize screen space and layout hierarchy:

1. **Asymmetric Split-Pane Layouts:** Instead of overloading users with nested navigation layers, the dashboards utilize flat, non-overlapping sidebars alongside wide monitoring areas. For example, in the receptionist view, form controls are clustered into a fixed-width left column (`prefWidth="280"`), leaving the remaining space available for the live queue tracking table.

2. **Dynamic Component Scaling and Growth:** Layout elements use responsive alignment constraints. Main presentation tables use the `VBox.vgrow="ALWAYS"` parameter to capture vertical space dynamically when windows scale. Column widths use a constrained policy (`CONSTRAINED_RESIZE_POLICY`) to prevent horizontal scrollbars and keep layout boundaries clean on diverse terminal displays.

3. **High-Contrast Visual Hierarchies:** Interfaces enforce clean data separation using dropshadow rendering effects, light background layers (`#f8f9fa`), and explicit borders to divide actions from data. Information cards capture immediate attention by using size-scaled typography to separate headers from secondary status logs.

5.2 Key Interface Showcase

The interface deployment is split across four core operational views designed to accommodate specific workflow tasks within the medical system.

5.2.1 Authentication Screen

The login interface represents the primary gateway to system resources. To ensure an immediate, clean user experience, the view uses a centered layout grid wrapped inside a responsive vertical container (`VBox`).

The authentication layout incorporates clear user feedback mechanisms:

- **Structured Entry Controls:** Captures user profiles using dedicated text inputs (`emailField`) and native password fields (`passwordField`).

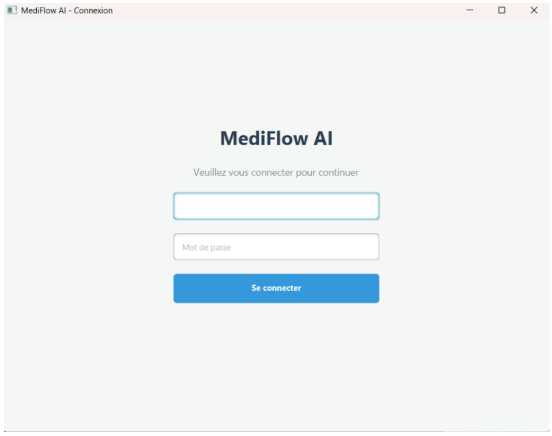

Figure 5.1: MediFlow AI Secure Authentication Interface

- **Reactive Status Labels:** Features an embedded status component (`errorLabel`). The system uses reactive CSS injection to change message styles based on state—flashing bold red text for authentication failures and bright green messages when credentials match successfully.

5.2.2 Receptionist Dashboard

The receptionist dashboard represents the operational nerve center for patient check-ins and clinical room mapping. This layout groups multiple system tools into a single view.

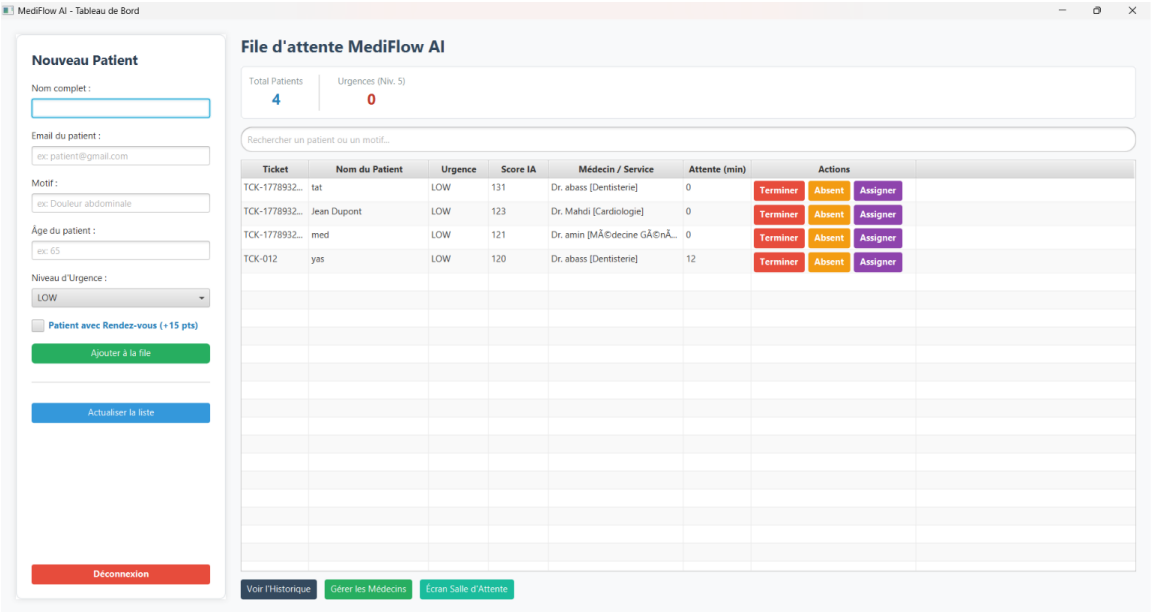

Figure 5.2: Receptionist Operational Management Dashboard Workspace

The workspace incorporates three specialized UX features:

- **Edge NLP Triage Assistant:** Features a live listener bound directly to the complaint input text (`reasonInput`). When critical keyword tokens are detected, the system overrides defaults to select `HIGH` urgency automatically, applying a vibrant red border to the input component to alert the receptionist.

- **Secure Dynamic Doctor Assignment ComboBox:** Replaces legacy ID typing inputs with an asynchronous dropdown selector. When the receptionist opens the menu, it queries active backend resources to build a clean picker list (e.g., *"Dr. Mahdi [Cardiologie]"*), preventing database linkage errors.
- **Conditional Table Highlights:** The table utilizes custom row factories. Individual patient rows change style based on calculated triage metrics, using high-visibility warning colors to ensure staff can easily distinguish urgent cases.

5.2.3 Practitioner Consultation Panel

The doctor consultation interface provides a minimal, distraction-free environment tailored to patient care and room tracking.

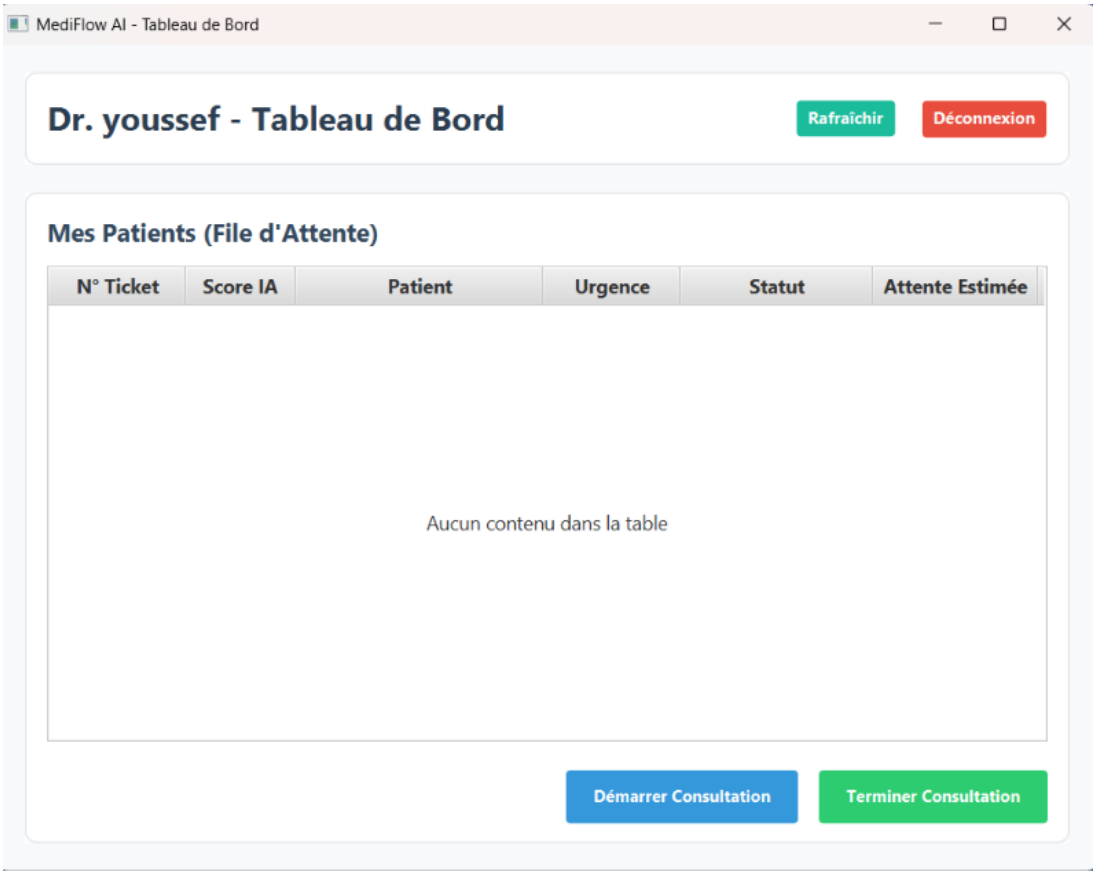

Figure 5.3: Medical Practitioner Private Consultation Queue Interface

Key design choices for the practitioner panel include:

- **Isolated Queue Views:** The view extracts the authenticated provider ID from the singleton context (`SessionContext`), filtering tables to show only the patients explicitly assigned to that office.
- **10-Second Automated Background Polling:** An background exececutor runs a sync loop every 10 seconds, keeping rows updated with incoming reception assignments without freezing active screen tasks.

- **Linear Action Buttons:** Large action buttons regulate care states, allowing doctors to transition status labels smoothly from waiting to active or complete with a single click.

5.2.4 Public Lounge Waiting Room Monitor

The public waiting room monitor is an optimized, high-visibility interface designed for large screens and televisions in clinical lounges.

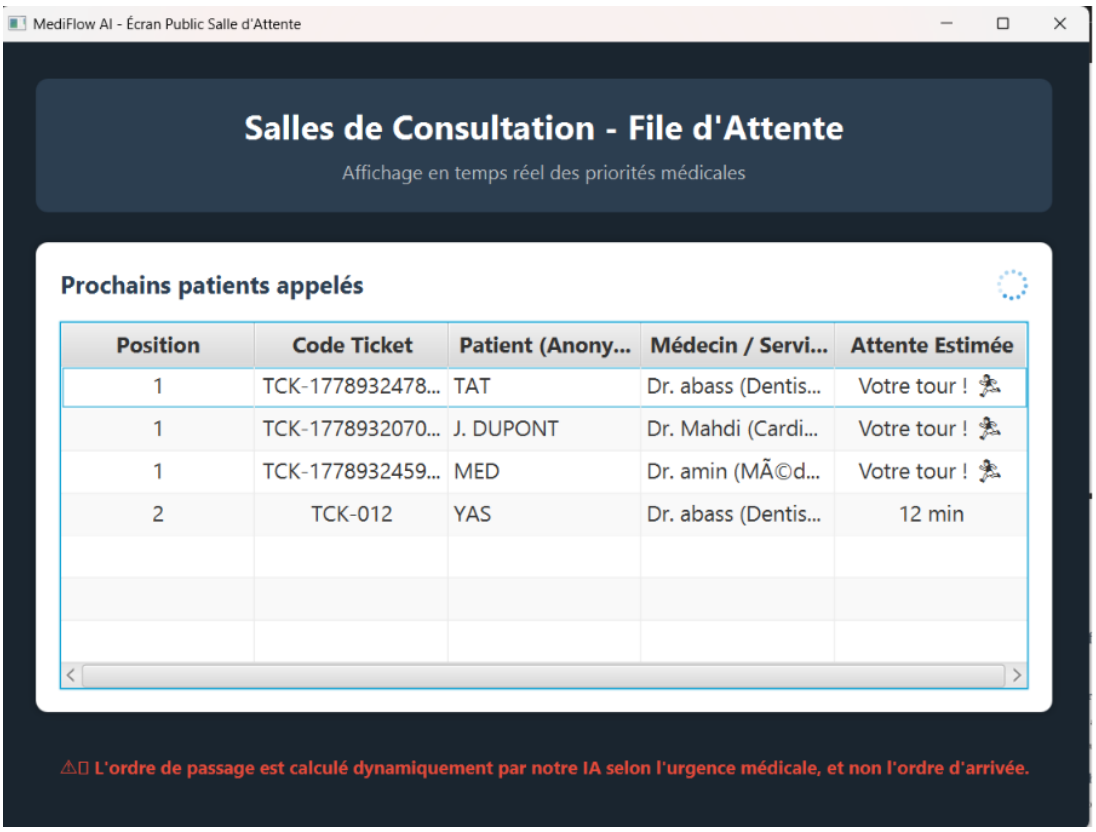

Figure 5.4: Read-Only Public Lounge Monitor Interface View

The public display implements strict privacy and layout constraints:

- **Strict Read-Only Architecture:** All input fields, context menus, and interactive options are completely omitted. Tables update silently using background threads to ensure data security in public areas.
- **GDPR-Compliant Data Anonymization:** The table controller uses a regex parsing engine to mask patient identities. Full name records are converted into clear, protected initials (e.g., *"Amine Slimani"* becomes `A. SLIMANI`), meeting data protection standards.
- **Clear Position Indexing:** Renders large, high-contrast numbers for position counters and wait times, ensuring data is easy to read from a distance.

6. Core Logic Engines & Notification Dispatches

6.1 Heuristic Priority Scoring Algorithm

To resolve the limitations of traditional First-In-First-Out (FIFO) clinical queues, **MediFlow AI** incorporates an automated multi-factor priority scoring engine within the backend service layer. The scheduling framework uses a dynamic mathematical heuristic model that balances acute medical needs with structured wait time safeguards.

Every incoming patient ticket is assigned an evolutionary score computed in real time by the `PriorityService` engine using the following formula:

$$\text{PriorityScore} = \text{UrgencyPoints} + \text{AppointmentBonus} + \text{VulnerabilityFactor} + \text{WaitTimeWeight}$$

The composite scoring system weights four specific tactical parameters:

- **Urgency Points (UrgencyPoints):** Mapped directly from the initial triage acuity tokens parsed during reception check-in. Critical cases receive heavy baseline values to push them to the front of the queue: `HIGH` updates the score by +60 points, `MEDIUM` adds +30 points, and `LOW` appends +10 points.
- **Appointment Bonus (AppointmentBonus):** A fixed value of +15 points is awarded to individuals with pre-scheduled calendar windows. This bonus balances unexpected critical walk-ins with structured appointments, ensuring that pre-booked appointments are respected.
- **Vulnerability Factor (VulnerabilityFactor):** Provides systemic demographic protection for fragile patient populations. The system parses the verified age string; if the value is greater than 60, it automatically adds a protective weight of +10 points to accelerate access to care.
- **Anti-Starvation Wait-Time Weight (WaitTimeWeight):** Solves the low-acuity starvation dilemma where non-critical patients face infinite delays during high-volume periods. The scheduler tracks wait times linearly, continuously appending +1 point for every 5 minutes elapsed since the ticket's creation timestamp. This ensures that any ticket will eventually reach the front of the queue regardless of its initial urgency tier.

6.2 Cloistered Queue Isolation Mechanics

A common flaw in generic hospital queue configurations is treating the facility as a single line, which creates routing bottlenecks if individual doctor workloads vary. **MediFlow AI** solves this by implementing cloistered queue isolation mechanics inside the `TicketService.java` execution layer.

When the receptionist assigns a patient to a doctor using the responsive `ComboBox` selector, the entry remains in a `WAITING` state but is tied to a specific practitioner ID. The prioritization engine then partitions the global waitlist into independent sub-queues.

```
// Extract all active waiting rows from the repository
List<Ticket> waitingTickets = ticketRepository.findByStatus("WAITING");

// Recalculate live priority scores across all active elements
waitingTickets.forEach(priorityService::calculateAndSetScore);

// Partition the global stream into cloistered sub-queues grouped by Doctor ID
Map<Long, List<Ticket>> ticketsByDoctor = waitingTickets.stream()
.collect(Collectors.groupingBy(t -> t.getDoctor() != null ? t.getDoctor().getId() : -1L));

// Compute relative metrics independently within each doctor's queue boundary
ticketsByDoctor.forEach((doctorId, doctorTickets) -> {
  List<Ticket> sortedDoctorTickets = doctorTickets.stream()
  .sorted(Comparator.comparingInt(Ticket::getPriorityScore).reversed()
  .thenComparing(Ticket::getCreatedAt))
  .collect(Collectors.toList());

  for (int i = 0; i < sortedDoctorTickets.size(); i++) {
    Ticket t = sortedDoctorTickets.get(i);
    t.setPositionNumber(i + 1); // Isolated sequential position within this office

    int avgTime = (t.getDoctor() != null && t.getDoctor().getService() != null)
    ? t.getDoctor().getService().getAverageConsultationTime() : 10;

    t.setEstimatedWaitingTime(i * avgTime); // Wait time cloisonne per practitioner
    ticketRepository.save(t);
  }
});
```

Listing 6.1: Cloistered Queue Grouping and Wait-Time Partitioning Logic

This partitioning strategy provides two core benefits:

1. **Parallel Flow Execution:** Because wait-time calculations are isolated per doctor (Position × AverageConsultationTime), patients assigned to different doctors advance independently. Multiple consultations can execute simultaneously without one slow room blocking other active rows.

2. **Data Consistency Across Interfaces:** The partitioned data structures feed both the doctor's private workbench and the public waiting room display. This ensures patients can see exactly which doctor they are waiting for, making the prioritized sequence transparent and reducing waiting room anxiety.

6.3 Automated Asynchronous Notification Systems

Clear, proactive communication is essential to managing large waiting areas effectively. The system implements a decoupled, automated notification subsystem using Java Mail Sender and Jakarta Mail protocols over secure SMTP transport channels.

6.3.1 Instant Intake Confirmation Ticket

Immediately following patient check-in and database persistence, the system triggers the creation of a sequential, human-readable usage shortcode. The system saves the record to capture the auto-incrementing primary key and formats it into a clean shortcode layout (e.g., ID 5 becomes `TCK-005`).

Once the identifier is confirmed, an asynchronous thread calls `emailService.sendInitialEmail(updatedTicket)`. This process formats an email notification containing the sequential shortcode, the assigned clinic division, the initial position in line, and the estimated wait time in minutes. Routing this heavy network operation through independent worker pools prevents the main receptionist interface from freezing during SMTP handshakes.

6.3.2 Proximity Cron Thread Alert

To ensure patients do not miss their call if they step away from the immediate lounge area, the backend configures an automated proximity alerting loop. This process uses Spring's scheduling context via the `@Scheduled(fixedRate = 30000)` annotation to scan active queue records every 30 seconds.

Figure 6.1: Asynchronous Cron Detection and SMTP Proximity Alert Pipeline

The background thread evaluates remaining wait times dynamically. When a patient's estimated wait drops to ≤ 5 minutes, the background cron task generates a priority reminder message. To maintain transaction reliability and prevent sending duplicate emails during subsequent 30-second execution loops, the process uses state flags (`reminderEmailSent = true`) to guarantee strict notification idempotency.

7. Error Handling & Operational Validation

7.1 System-Wide Error Handling Architecture

To operate reliably within a high-throughput medical environment, **MediFlow AI** implements a defensive error-handling architecture across both its frontend presentation and backend persistence tiers. This dual-boundary architecture prevents fatal application crashes, protects data integrity during concurrent writes, and guarantees clear visual feedback when unexpected system errors occur.

7.1.1 Network Boundary Fallbacks

Because the platform relies on a decoupled client-server model, the JavaFX desktop application must gracefully handle unexpected network failures, such as server offline states, connection timeouts, or malformed JSON responses.

To achieve this, all data transactions routing through the remote consumers (e.g., `AuthApiService`, `DoctorApiService`) are wrapped inside robust operational exception boundaries:

- **Asynchronous Exception Catching:** Network requests processed via `java.net.http.HttpClient` execute within asynchronous threads. The frontend utilizes functional callback streams—such as `.exceptionally()` blocks—to intercept lower-level socket dropouts (e.g., `ConnectException`, `HttpTimeoutException`) before they propagate to the main execution loop.
- **Non-Blocking Error Modal Dialogs:** When a network failure is caught, the system prevents interface freezing by redirecting execution back to the main UI thread via `Platform.runLater()`. It displays a user-friendly, descriptive modal dialog using the native JavaFX `Alert(AlertType.ERROR)` component, prompting the receptionist or doctor to retry the action safely.
- **Deterministic HTTP Status Auditing:** The client-side handlers explicitly parse HTTP response headers. If the server passes error response codes (such as `401 Unauthorized` for wrong credentials or `500 Internal Server Error` for database bottlenecks), the client decodes the error payload cleanly, displaying tailored contextual instructions instead of raw terminal trace logs.

7.1.2 Database Integrity Checks

On the backend server side, preventing data corruption across relational structures requires strict control over concurrent table access, especially when multiple operators update the queue simultaneously. The application enforces complete relational safety using automated architectural guardrails:

- **Atomic Transaction Boundaries:** Critical multi-step workflows—such as creating a ticket, generating its sequential shortcode, and modifying position arrays—are bound by Spring's declarative transaction manager using the `@Transactional` annotation layer. If an unexpected error occurs during any step (e.g., an SMTP mail-relay failure), the entire database state rolls back automatically to its last valid savepoint, preventing corrupted or orphan entries in the `queue_tickets` table.
- **Unique Key Constraints and Index Mappings:** The underlying MySQL database schema forces strict structural constraints. Data inputs are validated against field boundaries, including unique index controls (`UK`) on user login metrics and foreign key rules (`FK`) joining patient files to active tickets. This prevents duplicate entry creation or broken data relations at the persistence tier.

—

7.2 Integration Testing Scenarios

To validate the end-to-end integration of the system—specifically verifying the cloistered queue isolation mechanics and the multi-factor priority scoring algorithm—a formal operational test script was executed.

7.2.1 Test Environment Blueprint

The simulation configuration was established by registering two distinct medical practitioners within the database layer:

- **Practitioner A:** `Dr. Mahdi` assigned to the **Cardiology** division (Average Consultation Time: 10 minutes).
- **Practitioner B:** `Dr. amin` assigned to the **Médecine Générale** division (Average Consultation Time: 10 minutes).

7.2.2 Operational Test Execution Script

Scenario Step 1: Initial Triage Check-In

- **Action Executed:** The receptionist signs in and registers an incoming walk-in patient: `Jean Dupont` (Age: 30, Urgence: `LOW`, No Appointment).
- **System Response & Log Verification:** The backend automatically processes the record. It skips the age and appointment bonuses, assigning a baseline priority score of 10 points. It successfully converts the record into sequential shortcode format, assigning it identifier `TCK-001` with status `WAITING` and mapping its doctor field to `En attente d'affectation [Triage général]`.

Scenario Step 2: Targeted Doctor Assignment

- **Action Executed:** The receptionist triggers the dropdown assignment box on `Jean Dupont`'s record row, selects `Dr. Mahdi [Cardiologie]`, and clicks the update action.

- **System Response & Log Verification:** The record updates instantly across the network. Both the receptionist control dashboard and the read-only waiting lounge monitor display the updated assignment field. Because Jean is currently the only patient assigned to Dr. Mahdi's cloistered sub-queue, the system sets his local queue position to 1 and drops his estimated waiting time to 0 minutes.

Scenario Step 3: High-Acuity Flow Parallelism Verification

- **Action Executed:** A second patient, `Amine Slimani` (Age: 68, Urgence: `HIGH`, Has Appointment), checks into the facility. The edge NLP triage listener captures his symptoms, automatically suggesting a high urgency status. The receptionist assigns him immediately to `Dr. amin [Médecine Générale]`.

- **System Response & Log Verification:** The mathematical engine processes the new entry, adding multiple weights (60 acuity + 15 appointment + 10 senior vulnerability) to compute a high priority score of 85 points.

  Even though Amine's priority score (85) is significantly higher than Jean's score (10), the system's cloistered queue mechanics keep the paths separate. Amine becomes position 1 within Dr. amin's general medicine queue, while Jean maintains position 1 within Dr. Mahdi's cardiology queue. Consequently, both patients display an estimated wait time of 0 minutes simultaneously, proving that independent sub-queues run in parallel across separate offices without blocking each other.

Scenario Step 4: Live Consultation Synchronization Lifecycle

- **Action Executed:** `Dr. Mahdi` logs into his private workspace view. The filtered table displays only `Jean Dupont`. The doctor clicks the *"Démarrer Consultation"* action.

- **System Response & Log Verification:** The ticket transitions to an active `IN_CONSULTATION` state. The 10-second background polling cycle automatically syncs this status change across the network. On the public lounge monitor display, Jean's row changes cleanly, updating his wait time metric to a flashing highlight reading `Votre tour !`, providing clear, real-time feedback to the waiting lounge.

Conclusion & Perspectives

Project Achievements Summary

The development of **MediFlow AI** represents a successful integration of decoupled client-server architecture, real-time algorithmic data processing, and user-centric interface engineering. By designing and deploying this system, the traditional, outdated First-In-First-Out (FIFO) queue models often found in clinical settings have been replaced by a dynamic operational framework.

Throughout this project, several key software engineering milestones were achieved:

- **Robust Architectural Decoupling:** Successfully separated concerns by deploying a stateless Spring Boot REST API core alongside a multi-threaded JavaFX rich desktop consumer application.
- **Dynamic Prioritization Logic:** Implemented a mathematical priority scoring heuristic that weights patient urgency, scheduling parameters, and demographic vulnerabilities, while utilizing an active anti-starvation mechanism to prevent unfair waiting times.
- **Cloistered Flow Isolation:** Developed a queue-partitioning engine that isolates patient flows by assigned practitioner, enabling parallel room processing and highly accurate wait-time estimations.
- **Responsive UX/UI Integration:** Crafted user-centric dashboards featuring real-time client-side NLP triage keyword evaluation, non-blocking asynchronous UI rendering (`Platform.runLater`), and GDPR-compliant patient name masking for public displays.
- **Asynchronous Automation Systems:** Automated notification pipelines using background Spring Schedulers and SMTP mail relays to distribute initial digital tickets and proactive proximity alerts seamlessly.

Future Roadmap & Perspectives

While the current implementation fulfills all functional and non-functional requirements, the system's modular architecture lays a strong foundation for future enterprise-grade upgrades and expansions:

1. **Advanced LLM Triage Agents:** Transition from localized, keyword-based text property listeners to advanced Agentic AI workflows. Integrating localized Large Language Models (such as a fine-tuned Llama 3.1 instance) via orchestration frameworks like LangGraph would enable deep semantic extraction of patient complaints, generating clinical risk evaluations with higher accuracy.

2. **Enterprise Security Hardening:** Upgrade the authentication layer from plaintext filtering to state-of-the-art secure transmission protocols. This includes introducing salted BCrypt password encoding on the persistence layer and issuing short-lived, stateful JSON Web Tokens (JWT) to secure API calls across network boundaries.

3. **Advanced Embedded Analytics Engine:** Embed an administrative performance dashboard leveraging modern embedded data engineering tools like DuckDB or specialized BI components. This would allow hospital administrators to perform sub-millisecond analytical queries on historical datasets, identifying operational bottlenecks, calculating average treatment costs, and tracking clinician efficiency cycles.

4. **Cross-Platform Mobile Integration:** Expand the client network by developing native mobile extensions. This would allow patients to check into the clinic remotely, receive live push notifications, and monitor their cloistered queue position from their smartphones before physically entering the facility.

In conclusion, **MediFlow AI** demonstrates how modern software architecture can significantly improve operational efficiency and patient care within healthcare IT. The platform balances technical stability with clean design, establishing a solid foundation for building smarter, scalable, and highly responsive clinical workflows.